

# R básico para ciencia de datos - Clase 1

Ph D. Stephanie Hereira-Pacheco

Universidad Autónoma de Tlaxcala

2026-05-19

- **¿Qué es R y por qué usarlo?**
- **RStudio y paneles**
- **Configuración del ambiente de trabajo y Scripts**
- **Directorios y sesión**
- **Áreas de trabajo y Proyectos**
- **Instalación y carga de paquetes**
- **Tipos de objetos, tipos de datos y estructuras de datos**
- **Funciones**

## ¿Qué es R?

- R es un lenguaje de programación **enfocado en estadística y análisis de datos**
- Fue creado por estadísticos como un ambiente interactivo para el análisis de datos
- Puede guardar su trabajo como una secuencia de comandos (*script*) que se puede ejecutar fácilmente en cualquier momento
- Otros programas (C, Java, Python) también pueden hacer cálculos estadísticos, pero R fue diseñado específicamente para eso

## Un poco de historia...

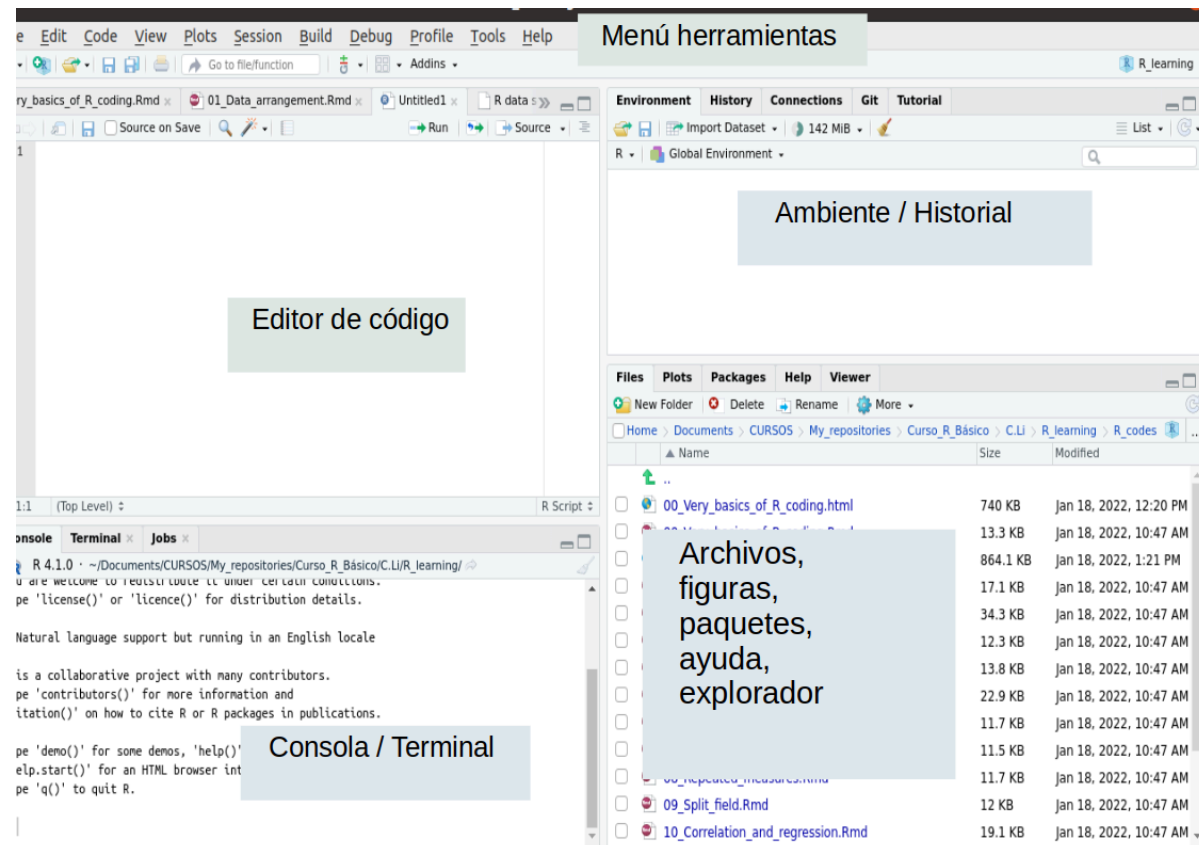
- R proviene del lenguaje **S**, creado en los Laboratorios Bell (EE.UU.) — los mismos que inventaron el transistor, el láser y Unix
- **Ross Ihaka** y **Robert Gentleman** (Universidad de Auckland, NZ) crearon una implementación libre y gratuita de S
- El trabajo inició en **1992** y la primera versión estable llegó en **2000**
- Hoy es mantenido por el *R Development Core Team*, con especialistas de todo el mundo
- R tiene **Licencia Pública GNU** — puede usarse para fines personales, académicos o comerciales sin restricciones

## ¿Por qué usar R?

1. Gratuito y de código abierto
2. Corre en Windows, Mac OS y UNIX/Linux — scripts compatibles entre plataformas
3. Comunidad grande, creciente y activa
4. Acceso temprano a nuevas metodologías publicadas como paquetes
5. Miles de paquetes en **CRAN**, **GitHub** y **Bioconductor**: ecología, biología molecular, ciencias sociales, geografía...

# RStudio

RStudio es nuestra plataforma de trabajo. Provee un editor visual e interactivo para crear y editar *scripts*, además de otras herramientas útiles.



## Paneles de RStudio

### **Panel superior izquierdo**

Editor de código — aquí se escriben y editan los scripts

### **Panel superior derecho**

*Environment* e *History* — objetos activos e historial de scripts

### **Panel inferior izquierdo**

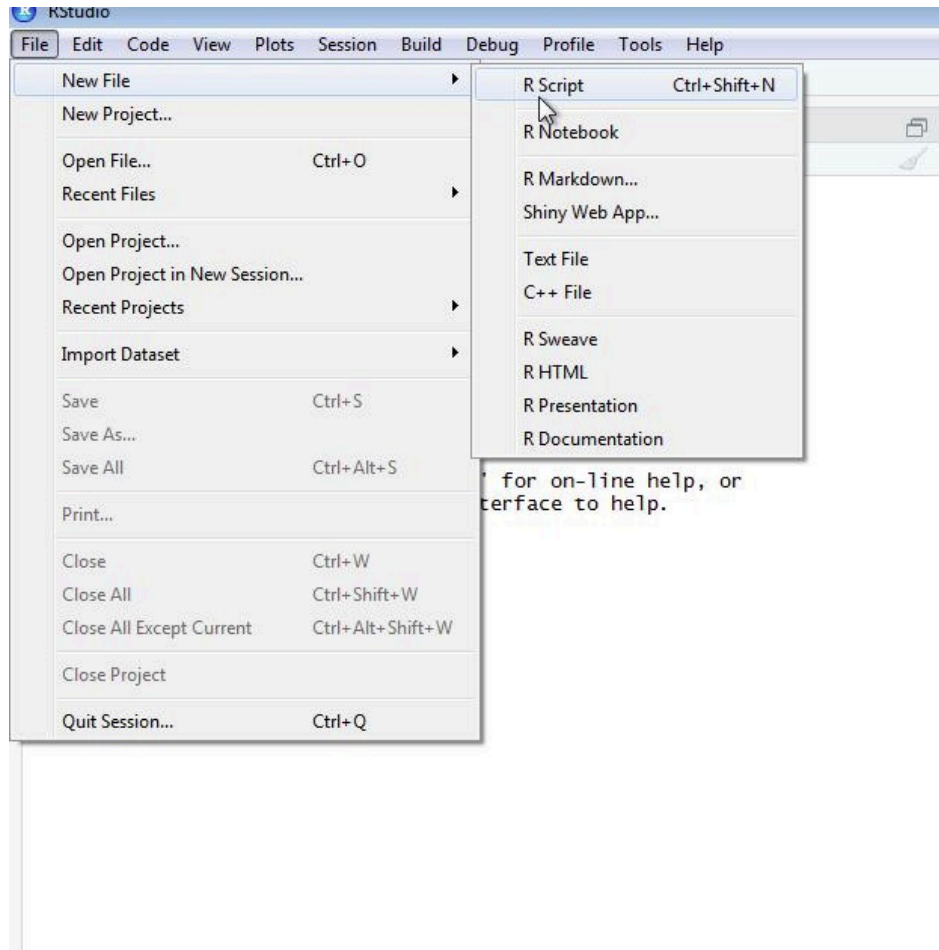
Consola de R — donde se ejecutan los códigos

### **Panel inferior derecho**

*Files* · *Plots* · *Packages* · *Help* · *Viewer*

# Scripts

Una de las grandes ventajas de R es que los códigos e instrucciones se guardan en *scripts*, que se pueden editar y reutilizar.





## Estructura recomendada de un *script*

```
1 # Título / descripción del script
2
3 library(datasets)    # primero: carga de paquetes
4 data(iris)           # segundo: carga de datos
5
6 summary(iris)        # análisis
7 boxplot(iris)        # gráficas
```

- El símbolo `#` indica un comentario — R lo ignora al ejecutar
- Carguen siempre paquetes y datos **al principio** del script
- Nombres de archivos: minúsculas, sin espacios → `mi_script.R`

## Abriendo un *script*

Pueden abrir y ejecutar scripts en R con la función `source()`:

```
1 source("C:/mi_script.R")
```

O bien: *File* → *Open File* y buscar en sus carpetas.

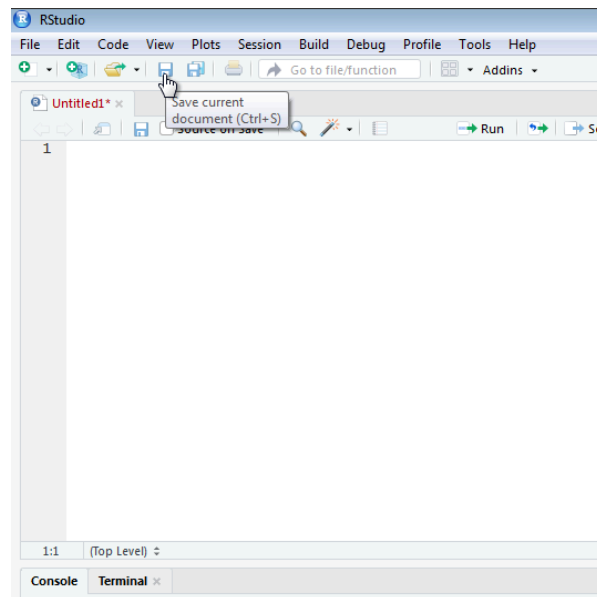
### Comentarios

---

Siempre inicien con comentarios descriptivos. Usen `#` para que R los ignore:

```
1 # Este es mi código  
2 data(iris)  
3 # Aquí termina el código
```

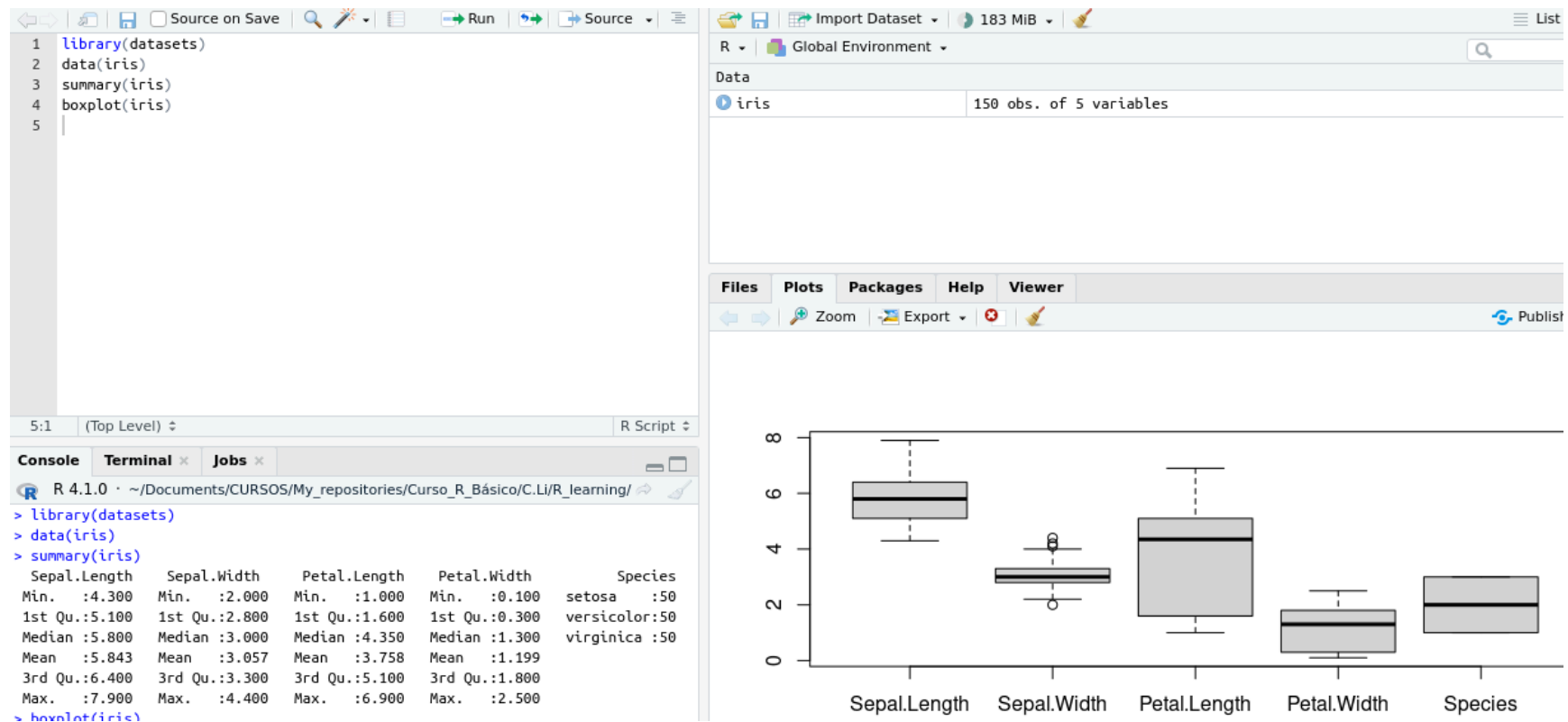
# Cómo ejecutar comandos



- Ejecutar línea actual: `Ctrl + Enter` (Windows/Linux) · `Cmd + Return` (Mac)
- Ejecutar todo el script: botón *Run* (esquina superior derecha del editor)
- Guarden con un nombre descriptivo, ej. `mi_script.R`

# Ejemplo: corriendo el código

```
1 library(datasets)
2 data(iris)
3 summary(iris)
4 boxplot(iris)
```



## Directorio de trabajo

El *directorio de trabajo* es el lugar donde R busca archivos para importar y donde guarda los resultados.

```
1 getwd()           # ver directorio actual
2 setwd("/home/steph/Desktop/") # establecer directorio
3 list.files()       # archivos en el directorio
4 list.dirs()        # carpetas en el directorio
```



Tip

**Mejor práctica:** usar **Proyectos de R** en lugar de `setwd()` — el directorio se configura automáticamente al abrir el proyecto.

## Sesión de R

Los objetos y funciones de R se almacenan en la **memoria** de la computadora. Cada vez que ejecutamos R, iniciamos una *sesión*.

- Los objetos creados permanecen solo en esa sesión
- Las sesiones no se comparten entre sí
- Al cerrar R, puede guardarse todo en un archivo **.Rdata**

```
1 ls()    # lista todos los objetos en la sesión actual
```

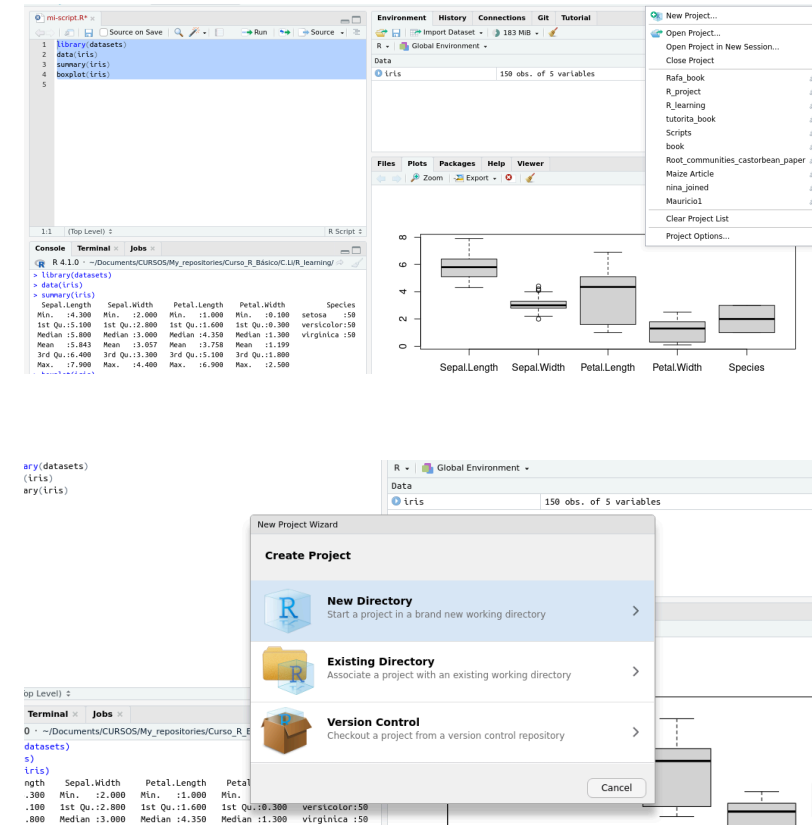


### Warning

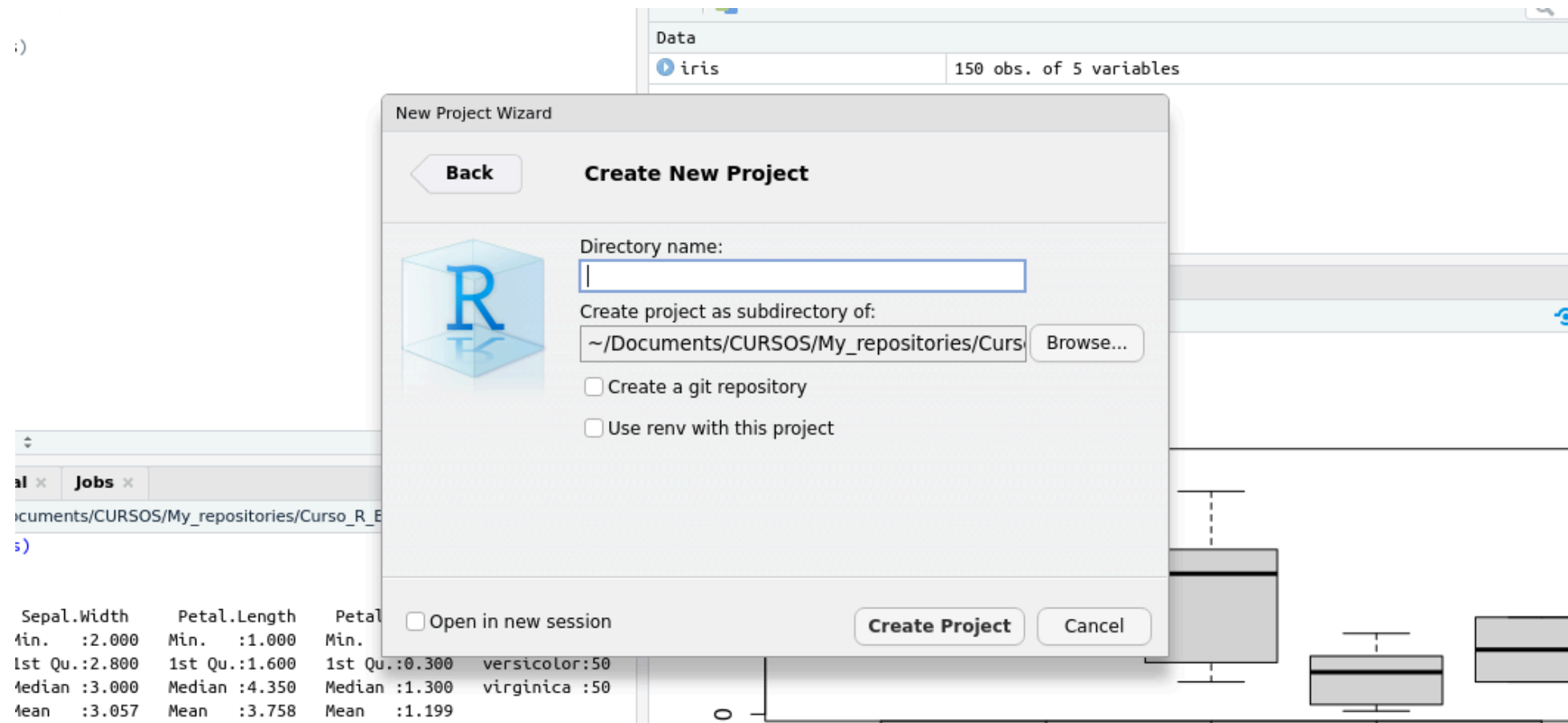
No se recomienda guardar el espacio de trabajo completo — genera confusión entre proyectos y ocupa mucho espacio en disco. Mejor exportar solo lo necesario.

# Proyecto de R (.Rproj)

- Identifica todos los archivos y contenido asociados a un trabajo
- Al crear un proyecto, las carpetas quedan **vinculadas directamente** a él
- El directorio de trabajo se establece automáticamente
- Un proyecto por cada curso, artículo o análisis



# Crear un proyecto





## Instalación de paquetes

- Muchos desarrolladores crean *paquetes* con funcionalidades adicionales
- Disponibles en **CRAN**, **GitHub** y **Bioconductor**

```
1 install.packages("tidyverse") # instalar desde CRAN
```

En RStudio: pestaña *Packages* → *Install*. Para cargar en cada sesión:

```
1 library(tidyverse)
```

### Important

Los paquetes se instalan **una sola vez**, pero se cargan **en cada sesión nueva**.

## Instalación: versiones de desarrollo y dependencias

Para paquetes en GitHub (versión en desarrollo):

```
1 devtools::install_github("rstudio/rmarkdown")
```

Los `::` llaman la función de un paquete sin cargarlo permanentemente en la sesión.

### Note

**Dependencias:** instalar `tidyverse` instala varios paquetes a la vez. Al cargarlo con `library()`, también se cargan sus dependencias automáticamente.

## Tipos de objetos en R

La información en R se estructura en forma de objetos, visibles en el panel *Environment*:

```
1 a      <- 1                                # escalar
2 letra  <- "a"                              # caracter
3 b      <- c(1, 2, 3)                        # vector
4 c      <- matrix(1:10)                      # matriz
5 d      <- data.frame(Especie=c("A","B"), Long=c(1,2)) # dataframe
6 e      <- list(c(1:20), c(1:10))            # lista
```

## Definiendo variables

- Usamos `<-` para asignar valores a las variables
- También puede usarse `=`, pero **no se recomienda**: en R el `=` implica igualdad lógica y puede causar confusiones

```
1 a <- 1
2 a          # evaluar imprime el valor
```

```
[1] 1
```

```
1 print(a)    # forma explícita de imprimir
```

```
[1] 1
```

## Visualizando variables asignadas

Los objetos declarados aparecen en el panel **Environment** (superior derecho). Desde ahí pueden verse el tipo y tamaño de cada objeto, o hacer clic para explorar tablas y listas.

Si se intenta usar un objeto que no existe en la sesión, R devuelve un error:

```
Error: object 'f' not found
```

### Note

Con `ls()` se obtiene una lista con los nombres de todo lo guardado en la sesión actual.

## Tips para nombrar variables

### Sí

- Empezar con letra: `mi_var`
- Sin espacios: `longitud_total`
- Minúsculas + guión bajo `_`
- Nombres descriptivos y significativos

### No

- Números al inicio: `1especie`
- Con espacios: `mi dato`
- Guiones medios: `mi-dato`
- Sobrecribir funciones: `mean <- 5`
- Caracteres especiales: `- ; . @ ?`

## Guardar el espacio de trabajo

Los objetos permanecen en el espacio de trabajo hasta que finaliza la sesión. Al salir, R pregunta si desean guardarlo (archivo `.Rdata`).



### Warning

**No recomendado:** sin organización por proyectos, es muy difícil darle seguimiento a los objetos guardados y ocupa mucho espacio en disco.



### Tip

**Recomendado:** un proyecto por cada trabajo o tarea, decidiendo explícitamente qué exportar y qué no.

## Exportar objetos de R

```
1 library(readr)
2 write_tsv(d, "data.tsv")           # separado por tabs
3 write.table(d, "data.txt", sep="\t")
4 write_csv(d, "data.csv")           # separado por comas
```



## Exportar objetos: formato nativo de R

Para guardar objetos conservando su estructura en R:

```
1 saveRDS(d, "data.RDS")
```

Para cargar el objeto nuevamente:

```
1 readRDS("data.RDS")
```

### Note

A diferencia de los formatos de texto, `saveRDS` conserva la clase y estructura del objeto (factores, listas, etc.)

## Tipos de datos

La función `class()` determina qué tipo de objeto tenemos:

```
1 a <- 5
2 class(a)
```

```
[1] "numeric"
```

| Tipo     | En R                   | Ejemplo                                |
|----------|------------------------|----------------------------------------|
| Numérico | <code>numeric</code>   | <code>5.1</code>                       |
| Entero   | <code>integer</code>   | <code>4L</code>                        |
| Texto    | <code>character</code> | <code>"hola"</code>                    |
| Factor   | <code>factor</code>    | Bajo / Medio / Alto                    |
| Lógico   | <code>logical</code>   | <code>TRUE</code> , <code>FALSE</code> |
| Perdido  | <code>NA</code>        | <code>NA</code>                        |
| Vacío    | <code>NULL</code>      | <code>NULL</code>                      |

## Para tener en cuenta...

### Note

**character** — El tipo más flexible. Siempre entre comillas: `"texto"`. Puede contener letras, números, espacios y símbolos especiales.

### Warning

**factor** — Dato numérico representado con etiquetas (niveles). Los niveles se ordenan **alfabéticamente** por defecto. Puede causar confusión: a veces se comporta como carácter y otras veces no — fuente común de errores.

### Important

**NA vs NULL** — **NULL** aparece cuando R intenta recuperar algo que no existe. **NA** representa explícitamente un **dato faltante** y puede surgir de operaciones que no pudieron completarse.

## Coerción de datos

En R los datos pueden ser *coercionados* — forzados a transformarse de un tipo a otro:

```
1 a <- 5
2 as.character(a)
```

```
[1] "5"
```

```
1 as.factor("medio")
```

```
[1] medio
Levels: medio
```

| Función                     | Convierte a |
|-----------------------------|-------------|
| <code>as.integer()</code>   | Entero      |
| <code>as.numeric()</code>   | Numérico    |
| <code>as.character()</code> | Texto       |
| <code>as.factor()</code>    | Factor      |
| <code>as.logical()</code>   | Lógico      |

## Tipos de estructura de datos

Los datos se estructuran de distintas formas. `class()` también identifica la estructura:

**Homogéneas** (un solo tipo de dato)

- **Vector** — 1 dimensión
- **Matriz** — 2 dimensiones

**Heterogéneas** (tipos mixtos)

- **Lista** — 1 dimensión
- **Data frame** — 2 dimensiones

# Vectores

Colecciones de uno o más datos del **mismo tipo**. No es posible mezclar tipos:

```
1 colores <- c("red", "black", "blue")  
2 is.vector(colores)
```

```
[1] TRUE
```

```
1 class(colores)
```

```
[1] "character"
```

## Vectorización

Las operaciones aritméticas y relacionales se aplican a **cada elemento** del vector automáticamente:

```
1 un_vector <- c(1:10)
2 un_vector * 10
```

```
[1] 10 20 30 40 50 60 70 80 90 100
```

```
1 un_vector + 1
```

```
[1] 2 3 4 5 6 7 8 9 10 11
```

```
1 un_vector + un_vector
```

```
[1] 2 4 6 8 10 12 14 16 18 20
```

## Matrices y arreglos

- Vectores multidimensionales — deben contener un **solo tipo** de datos
- Una matriz tiene **2 dimensiones**; un arreglo (*array*) puede tener **n** dimensiones

```
1 matri <- matrix(1:20, nrow=3, ncol=4)
2 dim(matri)
```

```
[1] 3 4
```

```
1 matri
```

|      | [,1] | [,2] | [,3] | [,4] |
|------|------|------|------|------|
| [1,] | 1    | 4    | 7    | 10   |
| [2,] | 2    | 5    | 8    | 11   |
| [3,] | 3    | 6    | 9    | 12   |



## Operaciones con matrices

```
1 matri * 2
```

|      | [,1] | [,2] | [,3] | [,4] |
|------|------|------|------|------|
| [1,] | 2    | 8    | 14   | 20   |
| [2,] | 4    | 10   | 16   | 22   |
| [3,] | 6    | 12   | 18   | 24   |

## Transponer una matriz

Rotar la matriz 90° con la función `t()`:

```
1 t(matri)
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 1    | 2    | 3    |
| [2,] | 4    | 5    | 6    |
| [3,] | 7    | 8    | 9    |
| [4,] | 10   | 11   | 12   |

# Listas

Las listas son **heterogéneas**: cada elemento puede ser de distinto tipo o clase:

```
1 un_vector  <- 1:20
2 una_matriz <- matrix(1:20, nrow=2)
3 una_df     <- data.frame("numeros"=1:3, "letras"=c("a","b","c"))
4 una_lista  <- list("vec"=un_vector, "mat"=una_matriz, "df"=una_df)
5 str(una_lista)
```

List of 3

```
$ vec: int [1:20] 1 2 3 4 5 6 7 8 9 10 ...
$ mat: int [1:2, 1:10] 1 2 3 4 5 6 7 8 9 10 ...
$ df : 'data.frame':    3 obs. of  2 variables:
..$ numeros: int [1:3] 1 2 3
..$ letras : chr [1:3] "a" "b" "c"
```

## Coercionando entre estructuras

```
1 df_transpuesta <- t(una_df)
2 class(df_transpuesta)
```

```
[1] "matrix" "array"
```

La función `t()` cambia la clase a matrix. Para conservar el dataframe:

```
1 df_transpuesta <- as.data.frame(t(una_df))
2 class(df_transpuesta)
```

```
[1] "data.frame"
```

# Funciones

- Ya usamos varias funciones: `library()`, `write_csv()`..
- La **sintaxis** de R requiere paréntesis para evaluar funciones
- R incluye muchas funciones por defecto; otras vienen en paquetes

```
1 log(a)
```

```
[1] 1.609438
```

```
1 sqrt(a)
```

```
[1] 2.236068
```

```
1 args(log)
```

```
function (x, base = exp(1))  
NULL
```

## Funciones: argumentos

```
1 help("log")  
2 ?log
```

- Argumentos con `=` en la ayuda son **opcionales** — tienen valor por defecto
- Ejemplo: `log` usa `base = exp(1)` por defecto (logaritmo natural)

```
1 log(x=8, base=2) # argumentos por nombre
```

```
[1] 3
```

```
1 log(8, 2) # argumentos por posición – mismo resultado
```

```
[1] 3
```

En las funciones sí se usa `=`, no `<-`, para especificar argumentos.

## Funciones: operadores aritméticos (sin paréntesis):

```
1 a + a
```

```
[1] 10
```

```
1 a - 2
```

```
[1] 3
```

```
1 1 * pi
```

```
[1] 3.141593
```

```
1 2 / 3
```

```
[1] 0.6666667
```

```
1 4 ^ a
```

```
[1] 1024
```

## Funciones: crear las tuyas (1)

Es posible declarar funciones propias. Por ejemplo, si quisiéramos definir el promedio:

```
1 average <- function(x) { sum(x) / length(x) }  
2 x <- 1:100  
3 average(x)
```

```
[1] 50.5
```

```
1 mean(x)          # comparamos con la función de R base
```

```
[1] 50.5
```

Al compararlas obtenemos el mismo resultado.



## Funciones: crear las tuyas (2)

La estructura general de una función en R es:

```
1 nombre <- function(argumentos) {  
2   operaciones  
3 }
```

- Primero se declara el nombre y los argumentos
- Luego las operaciones que realizará
- Lo importante: hemos declarado un **nuevo objeto** de tipo función en nuestra sesión

## Referencias

- Irizarry, R.A. (2019). *Introduction to Data Science*. Leanpub. [rafalab.dfci.harvard.edu/dsbook](http://rafalab.dfci.harvard.edu/dsbook)
- Wickham, H. & Grolemund, G. (2017). *R for Data Science*. O'Reilly. [r4ds.had.co.nz](http://r4ds.had.co.nz)
- Bosco Mendoza, J. (2019). *R para principiantes*. [bookdown.org/jboscomendoza/r-principiantes4](http://bookdown.org/jboscomendoza/r-principiantes4)
- CRAN — Comprehensive R Archive Network: [cran.r-project.org](http://cran.r-project.org)
- RStudio Education: [education.rstudio.com](http://education.rstudio.com)